

Motivación y aplicaciones

Cecilia Hernández

Universidad de Concepción, Chile

Agosto, 2026

- Datos masivos en muchas aplicaciones (análisis de datos, tráfico de red, patrones en grafos, bioinformática, neurociencia, astronomía, machine learning, etc)
- Datos masivos que llegan continuamente, por ejemplo tráfico de red.
- En muchos casos, existen restricciones fuertes de memoria y tiempo de procesamiento por elemento.
- Respuestas aproximadas con garantías matemáticas pueden ser aceptadas.

Por qué estudiar streaming

- Muchos sistemas no conocen los datos con anticipación, sino como una secuencia continua, incluso puede ser infinita:

$$a_1, a_2, \dots, a_m.$$

- El tamaño total m puede ser enorme o desconocido.
- Guardar todos los datos puede ser imposible, caro o innecesario.
- La pregunta cambia desde “procesar toda la entrada” hacia mantener un resumen suficiente para algún fin.

Idea central

stream $\rightarrow$ resumen compacto $\rightarrow$ consulta aproximada

Ejemplos de aplicación

- **Redes:** flujos pesados, anomalías, conteo de usuarios o direcciones IP distintas.
- **Logs:** eventos frecuentes, cambios de comportamiento, detección de fallas.
- **Sensores:** agregación de mediciones y detección de eventos en tiempo real.
- **Genómica:** conteo aproximado de k-mers, similitud entre genomas, filtrado de pares candidatos. Aunque acá se pueden conocer los datos en forma anticipada y no desaparecen.
- **Machine Learning:** reducción de dimensionalidad, features compactas y aprendizaje sobre datos que cambian.

Definición formal del modelo de streaming

Un algoritmo de streaming recibe una secuencia de actualizaciones:

$$\sigma = u_1, u_2, \dots, u_m.$$

Cada actualización llega una vez y debe procesarse inmediatamente.

Estado compacto

El algoritmo no guarda todo el stream. Mantiene un estado pequeño S_t y aplica:

$$S_{t+1} = \text{Update}(S_t, u_{t+1}).$$

Luego responde consultas usando solamente S_t :

$$\hat{x} = \text{Query}(S_t).$$

La dificultad es diseñar S_t para que use poca memoria, se actualice rápido y conserve la información necesaria para la consulta.

Universo, stream y vector de frecuencias

Sea

$$\mathcal{U} = \{1, 2, \dots, n\}$$

el universo de elementos posibles. Un stream induce un vector de frecuencias:

$$\mathbf{f} = (f_1, f_2, \dots, f_n),$$

donde f_i representa cuántas veces aparece el elemento i .

Ejemplo

Si el stream es

$$A, B, A, C, A, B,$$

entonces

$$f_A = 3, \quad f_B = 2, \quad f_C = 1.$$

Muchos problemas de streaming se formulan como estimar una función de $\mathbf{f}$ sin almacenar $\mathbf{f}$ completo.

Tipos en base a actualizaciones

Insert-only o cash-register

Cada actualización incrementa una frecuencia $\Delta > 0$:

$$(i, \Delta), \quad f_i \leftarrow f_i + \Delta.$$

Las frecuencias nunca disminuyen.

Turnstile

Cada actualización puede sumar o restar ($\Delta \in \mathbb{Z}$). Permiten inserción y eliminación:

$$(i, \Delta), \quad f_i \leftarrow f_i + \Delta.$$

Restricciones del modelo

Un algoritmo de streaming se analiza usando restricciones explícitas.

- **Una o pocas pasadas:** idealmente el stream se procesa una sola vez.
- **Memoria sublineal:** mucho menor que almacenar toda la entrada ($o(n)$).
- **Tiempo de actualización bajo:** idealmente $O(1)$ o polylog por elemento.
- **Consulta eficiente:** responder usando solo el resumen compacto.
- **Exactitud o aproximación:** según el problema, se acepta error controlado.

Modelo básico

- El algoritmo observa cada elemento una vez, o pocas veces.
- Mantiene un estado pequeño S_t .
- Cuando llega a_{t+1} , actualiza:

$$S_{t+1} = \text{Update}(S_t, a_{t+1}).$$

- Al final, o durante el stream, responde una consulta:

$$\hat{x} = \text{Query}(S_t).$$

Restricción

El estado S_t debe ser mucho más pequeño que el stream completo.

Diseño y análisis de algoritmos en streaming

En streaming se analizan más dimensiones que en un algoritmo exacto tradicional.

- **Espacio:** memoria usada por el resumen.
- **Tiempo de actualización:** costo por elemento recibido.
- **Número de pasadas:** idealmente una pasada.
- **Error:** distancia entre la respuesta exacta x y la estimación $\hat{x}$.
- **Probabilidad de falla:** probabilidad de que el error exceda la garantía (en el caso sean algoritmos aleatorizados).
- Para algoritmos deterministas se necesita solo una garantía de error.
- Típicamente los aleatorizados requieren menos espacio de memoria para un mismo error.

Una garantía típica de Prob de falla es:

$$\Pr[|\hat{x} - x| \leq \epsilon B] \geq 1 - \delta.$$

Streaming describe principalmente un modelo de computación.

- Existen sketches construidos en forma determinista, por ejemplo usando contadores exactos. Ejemplos de sketches deterministas: Misra Gries y SpaceSaving.
- Muchos sketches son aleatorizados: usan hashing, muestreo o proyecciones aleatorias para reducir fuertemente la memoria.
- La aleatoriedad permite obtener garantías del tipo:

$$\Pr [|\hat{x} - x| \leq \varepsilon B] \geq 1 - \delta.$$

Qué es un sketch

Un sketch es una representación compacta de un stream, diseñada para estimar eficientemente una propiedad específica usando memoria sublineal.

- No busca reconstruir todos los datos originales.
- Busca responder consultas de manera aproximada.
- Usa poca memoria y actualizaciones rápidas.

Ejemplos de consultas

- Frecuencia de un elemento.
- Elementos más frecuentes.
- Número de elementos distintos.
- Percentiles o cuantiles.
- Similitud entre conjuntos o ventanas.

Memoria sublineal y polylog

Un objetivo común es usar memoria sublineal:

$$o(n),$$

es decir, menor que almacenar una estructura de tamaño proporcional a n .

Complejidad polylog

Una función es polylogarítmica si tiene la forma:

$$O((\log n)^k),$$

para alguna constante k .

Ejemplos:

$$O(\log n), \quad O(\log^2 n), \quad O(\log^3 n).$$

Ejemplo: Estimación de cuentas (o frecuencias)

Supongamos que queremos contar apariciones de elementos en un universo grande.

f_i = número de apariciones del elemento i .

Una solución exacta necesita mantener muchas frecuencias:

$$(f_1, f_2, \dots, f_n).$$

Un sketch mantiene una estructura pequeña y responde:

$$\hat{f}_i \approx f_i.$$

Este tipo de idea motiva algoritmos como Count-Min Sketch y CountSketch.

Los sketches pueden actuar como una etapa previa a un modelo de aprendizaje.

stream $\rightarrow$ sketch $\rightarrow$ features compactas $\rightarrow$ modelo ML.

Ejemplos:

- Reducción de dimensionalidad mediante hashing o CountSketch.
- Detección de anomalías usando frecuencias, cambios y cardinalidades aproximadas.
- Detección de concept drift comparando resúmenes de ventanas consecutivas.
- Clasificación de flujos, usuarios o eventos a partir de features compactas.

Machine Learning y sketches: relación bidireccional

Los sketches y Machine Learning pueden combinarse de dos formas distintas.

Sketch $\rightarrow$ ML

El sketch resume el stream y produce información compacta para el modelo:

$\text{stream} \rightarrow \text{sketch} \rightarrow \text{ML}$.

Ejemplos: frecuencias, cardinalidad, similitud y cambios entre ventanas.

ML $\rightarrow$ Sketch

El modelo aprendido ayuda a decidir cómo utilizar la memoria del sketch:

$\text{stream} \rightarrow \text{ML} \rightarrow \text{sketch}$.

Objetivo: reducir colisiones o mejorar la relación memoria y error.

Learned sketches

Un **learned sketch** incorpora predicciones aprendidas para mejorar una estructura de streaming tradicional o aprender una estrategia de compresión.

- Un modelo puede predecir **heavy hitters** y mantenerlos fuera del sketch; el resto se procesa con Count-Min u otra estructura compacta.
- También puede **aprender la asignación a buckets** para disminuir colisiones frecuentes.
- Los **neural sketches** aprenden cómo comprimir el stream bajo un presupuesto de memoria.

Learned sketches vs. Neural sketches

Learned sketches

- Mantienen una estructura de sketch tradicional.
- Un modelo de ML mejora alguna parte del algoritmo:
 - identificar *heavy hitters*;
 - decidir qué elementos almacenar;
 - reducir colisiones;
 - mejorar la estimación.
- La lógica principal del sketch sigue siendo diseñada explícitamente.
- ML produce mejora en sketch clásico.

Ejemplo: Learned Count-Min

Neural sketches

- Red neuronal aprende directamente parte importante del mecanismo de sketching.
- Una red con memoria puede aprender:
 - cómo representar el stream, cómo actualizar la memoria, qué información preservar, cómo responder consultas.
- Actualización del sketch depende de red neuronal.

Ejemplos: Meta-Sketch, Lego Sketch.

Un grafo también puede recibirse como stream de aristas:

$$(u_1, v_1), (u_2, v_2), \dots, (u_m, v_m).$$

En lugar de guardar todas las aristas, se puede mantener un resumen para estimar propiedades del grafo.

- Grado de vértices y aristas frecuentes.
- Componentes conectados en modelos semi-streaming.
- Conteo aproximado de triángulos.
- Sparsification para obtener un grafo reducido.
- Features de grafos para modelos de ML.

Sampling vs algoritmos de streaming

Sampling

- Mantiene solo una muestra de los elementos del stream.
- Reduce memoria descartando gran parte de los datos (no todos los elementos se procesan).
- El resultado depende de que la muestra sea representativa.
- Puede perder eventos poco frecuentes pero importantes.
- Ejemplo: reservoir sampling.

Stream $\longrightarrow$ muestra S

$$|S| \ll n$$

Algoritmos de streaming

- Mantienen un resumen compacto del stream.
- Cada elemento puede contribuir al resumen (todos los elementos son procesados, aunque no requiere conservar elementos).
- El resumen se diseña para responder consultas específicas.
- Pueden entregar garantías de error y probabilidad.
- Ejemplos: Count-Min, CountSketch, HLL, KLL.

Stream $\longrightarrow$ Sketch $\longrightarrow \hat{f}$

Algoritmos de streaming vs algoritmos online: no son lo mismo

Ambos procesan la entrada secuencialmente, pero enfatizan aspectos distintos.

Streaming	Online
Mantener poca información del pasado	Tomar decisiones sin conocer el futuro
Memoria sublineal o compacta	Decisiones irrevocables
Consultas estimadas	Asignaciones, selecciones o acciones
Error, espacio, pasadas	Calidad comparado con al óptimo
Ejemplo: estimar cargas con sketches	Ejemplo: asignar trabajos a servidores

Un algoritmo puede ser online y, al mismo tiempo, usar una estructura de streaming para mantener información aproximada del pasado.

Clustering escalable

En clustering masivo, ejecutar directamente un algoritmo sobre todos los datos puede ser muy costoso. Una estrategia es construir primero un resumen:

$$D \rightarrow C(D) \rightarrow \text{clustering}.$$

Donde $C(D)$ puede ser un coreset, micro-clusters o una representación compacta.

- BIRCH resume datos mediante estructuras compactas.
- CluStream separa una fase online de micro-clusters y una fase offline de clustering.
- Coresets permiten aproximar objetivos como k-means con menos puntos.
- MinHash puede apoyar clustering por similitud entre conjuntos grandes.

Ejemplos concretos usando sketches

Estimación de entropía en tráfico de red

$$H = - \left[\sum_{i=1}^K \frac{m_i}{M} \log \left(\frac{m_i}{M} \right) + \sum_{i=K+1}^N \frac{m_i}{M} \log \left(\frac{m_i}{M} \right) \right].$$

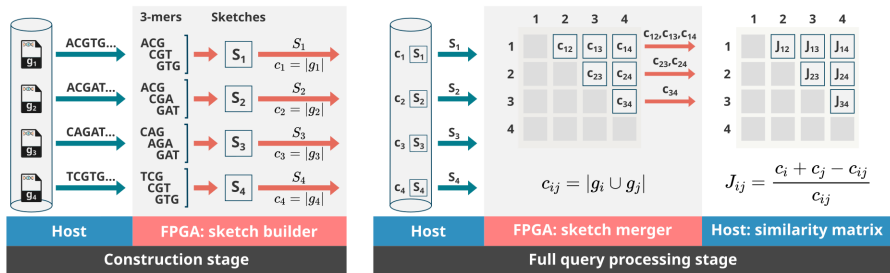
Estimación de entropía en tráfico de red (usando ajuste de powerlaw)

$$\hat{H} = \underbrace{- \sum_{i=1}^K \frac{\hat{m}_i}{M} \log \left(\frac{\hat{m}_i}{M} \right)}_{\hat{H}_1} + \underbrace{- \sum_{i=K+1}^{\hat{L}} \hat{C} i^{-\hat{\alpha}} \log \left(\hat{C} i^{-\hat{\alpha}} \right)}_{\hat{H}_2} + \underbrace{\frac{\hat{N} - \hat{L}}{M} \log M}_{\hat{H}_3}.$$

donde, Power law es distribución donde pocos elementos concentran frecuencias altas y muchos elementos presentan frecuencias bajas, siguiendo aproximadamente $p_i \propto i^{-\alpha}$.

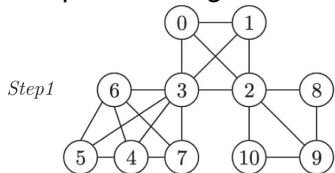
Ejemplos concretos usando sketches

Estimación de matriz de similitud genómica usando sketches



Ejemplos concretos grafos

Compresión de grafos mediante cliques superpuestos.



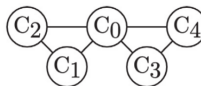
$C_0 : 0, 1, 2, 3$

$C_1 : 3, 4, 6, 7$

$C_2 : 3, 4, 5, 6$

$C_3 : 2, 8, 9$

$C_4 : 2, 9, 10$



Step2

$u \in G$	0	1	3	4	5	6	7	2	8	9	10
R_r	4.0	4.0	4.0	4.0	4.0	4.0	4.0	3.3	3.0	3.0	3.0

$CP: \begin{bmatrix} C_1 & C_2 & C_3 & C_4 & C_0 \end{bmatrix}$

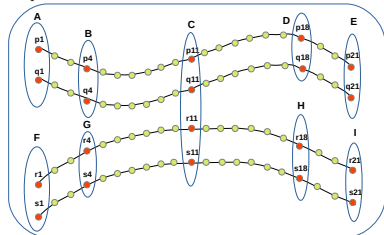
Step3

$X:$	3	4	5	6	7	2	8	9	10	0	1	2	3
$B:$	1	0	0	0	0	1	0	0	0	1	0	0	0
$BB:$	3	3	2	3	1	3	1	3	2				
$Y:$	0	5	9										

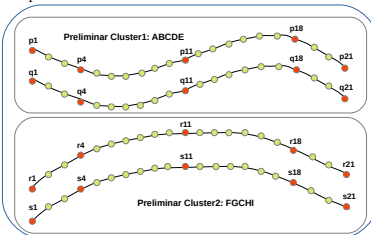
Ejemplos concretos: clustering eficiente

Clustering eficiente de fibras cerebrales.

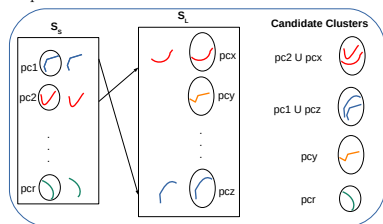
Step1



Step2



Step3



Step4

